



Dynamic Programming

Top Down & Bottom UP

$$c(n, k) = \begin{cases} c(n-1, k-1) + c(n-1, k) & \text{if } 0 < k < n \\ 1 & \text{if } k = 0 \text{ or } k = n \\ 0 & \text{otherwise} \end{cases}$$

Algorithm $c(n, k)$

if $k = 1$ **or** $k = n$

return 1

if $k < 0$ **or** $k > n$

return 0

return ($c(n-1, k-1) + c(n-1, k)$)

Top Down Recursive

$$c(n, k) = \begin{cases} c(n-1, k-1) + c(n-1, k) & \text{if } 0 < k < n \\ 1 & \text{if } k = 0 \text{ or } k = n \\ 0 & \text{otherwise} \end{cases}$$

N	0	1	2	3
0				
1				
2				
3				
4				
5				
6				20

◆ Recursively call the function with the smaller function.

◆ Time efficiency is tremendous!

$C(6,3)$

Top Down Memoization

$$c(n, k) = \begin{cases} c(n-1, k-1) + c(n-1, k) & \text{if } 0 < k < n \\ 1 & \text{if } k = 0 \text{ or } k = n \\ 0 & \text{otherwise} \end{cases}$$

N	0	1	2	3
0				
1	1	1		
2	1	2	1	
3	1	3	3	1
4		4	6	4
5			10	10
6				20

- ◆ Create the table to look up.
- ◆ Store the values that have been already calculated.

$C(6,3)$

Time efficiency : $\Theta(nk)$

Space efficiency : $\Theta(nk)$

Top Down Memoization

$$c(n, k) = \begin{cases} c(n-1, k-1) + c(n-1, k) & \text{if } 0 < k < n \\ 1 & \text{if } k = 0 \text{ or } k = n \\ 0 & \text{otherwise} \end{cases}$$

Algorithm *Lookup_c(n,k)*

if $k = 1$ **or** $k = n$

return 1

if $k < 0$ **or** $k > n$

return 0

if $c[n, k] < 0$ // < 0 means never been stored in and look up.

$c[n, k] \leftarrow \text{lookup_c}(n-1, k-1) + \text{Lookup_c}(n-1, k)$

return($c[n, k]$)

Bottom Up Memoization

$$c(n, k) = \begin{cases} c(n-1, k-1) + c(n-1, k) & \text{if } 0 < k < n \\ 1 & \text{if } k = 0 \text{ or } k = n \\ 0 & \text{otherwise} \end{cases}$$

N	0	1	2	3
0	1			
1	1	1		
2	1	2	1	
3	1	3	3	1
4	1	4	6	4
5	1	5	10	10
6	1	6	15	20

- ◆ Fill a value in the table from the smaller value.
- ◆ Once we found what we are looking for, then stop filling the table.

C(6,3)

Time efficiency : $\Theta(nk)$

Space efficiency : $\Theta(nk)$

Dynamic Programming

$$c(n, k) = \begin{cases} c(n-1, k-1) + c(n-1, k) & \text{if } 0 < k < n \\ 1 & \text{if } k = 0 \text{ or } k = n \\ 0 & \text{otherwise} \end{cases}$$

N	0	1	2	3
0				
1				
2				
3				
4				
5	1	5	10	10
6	1	6	15	20

- ◆ We don't need to remember all the value in the tables.
- ◆ We can reduce the space by remember only the value from the previous row and column.

$C(6,3)$

Top Down & Bottom Up

◆ Bottom up dynamic programming

- All sub-problems must be solved.
- Regular pattern of table access can be exploited to reduce time or space.

◆ Top Down + memoization

- Solve only sub problems that are definitely required.
- Recursion overhead.

Dynamic Programming

- ◆ Generally used for solving optimization problem that contains two characters
 - Optimal Substructure
 - Overlapping sub-problems
- ◆ Overlapping Sub-problems
 - Space of sub-problems must be “small” (Polynomial in the input size).
 - Recursive algorithm solves the same sub-problems over and over
 - Dynamic programming solves all sub-problems but solves each once and then stores the solution in some data structure.

Matrix Chain Multiplication

- ◆ **Dynamic Programming** is a general algorithm design paradigm.
 - Rather than give the general structure, let us first give a motivating example:

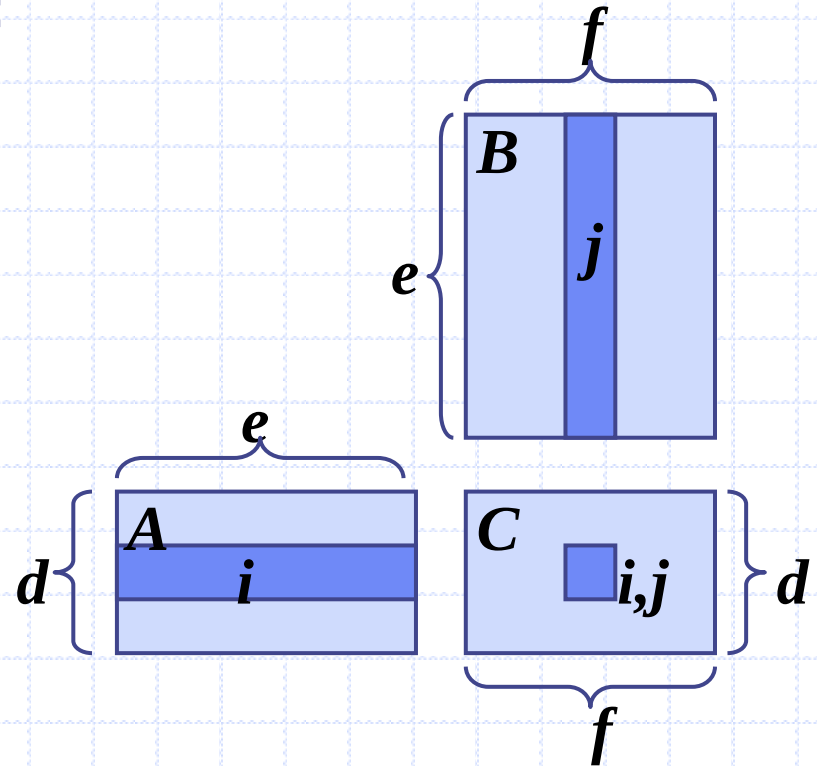
- **Matrix Chain-Products**

- ◆ Review: Matrix Multiplication.

- $C = A * B$
- A is $d \times e$ and B is $e \times f$

$$C[i, j] = \sum_{k=0}^{e-1} A[i, k] * B[k, j]$$

- $O(d * e * f)$ time



Matrix Chain Multiplication

◆ Matrix Chain-Product:

- Compute $A = A_0 * A_1 * \dots * A_{n-1}$
- A_i is $d_i \times d_{i+1}$
- Problem: How to parenthesize?

◆ Example

- B is 3×100
- C is 100×5
- D is 5×5
- $(B * C) * D$ takes $1500 + 75 = 1575$ operations
- $B * (C * D)$ takes $1500 + 2500 = 4000$ operations

An Enumeration Approach

◆ Matrix Chain-Product Algorithm:

- Try all possible ways to parenthesize $A=A_0*A_1*\dots*A_{n-1}$
- Calculate number of ops for each one
- Pick the one that is best

◆ Running time:

- The number of paranthesizations is equal to the number of binary trees with n nodes
- This is **exponential!**
- It is called the Catalan number, and it is almost 4^n .
- This is a terrible algorithm!

A Greedy Approach

◆ Idea#1: repeatedly select the product that uses (up) the most operations.

◆ Example:

- A is 10×5
- B is 5×10
- C is 10×5
- D is 5×10
- Greedy idea#1 gives $(A*B)*(C*D)$, which takes $500+1000+500 = 2000$ ops
- $A*((B*C)*D)$ takes $500+250+250 = 1000$ ops

Another Greedy Approach

- ◆ Idea #2: repeatedly select the product that uses the fewest operations.
- ◆ Example:
 - A is 101×11
 - B is 11×9
 - C is 9×100
 - D is 100×99
 - Greedy idea#2 gives $A*((B*C)*D)$, which takes $109989 + 9900 + 108900 = 228789$ ops
 - $(A*B)*(C*D)$ takes $9999 + 89991 + 89100 = 189090$ ops
- ◆ The greedy approach is not giving us the optimal value.

A “Recursive” Approach

◆ Define **sub-problems**:

- Find the best parenthesization of $A_i * A_{i+1} * \dots * A_j$.
- Let $N_{i,j}$ denote the number of operations done by this sub-problem.
- The optimal solution for the whole problem is $N_{0,n-1}$.

◆ **Subproblem optimality**: The optimal solution can be defined in terms of optimal sub-problems

- There has to be a final multiplication (root of the expression tree) for the optimal solution.
- Say, the final multiply is at index i : $(A_0 * \dots * A_i) * (A_{i+1} * \dots * A_{n-1})$.
- Then the optimal solution $N_{0,n-1}$ is the sum of two optimal sub-problems, $N_{0,i}$ and $N_{i+1,n-1}$ plus the time for the last multiply.
- If the global optimum did not have these optimal sub-problems, we could define an even better “optimal” solution.

A Characterizing Equation

- ◆ The global optimal has to be defined in terms of optimal sub-problems, depending on where the final multiply is at.
- ◆ Let us consider all possible places for that final multiply:
 - Recall that A_i is a $d_i \times d_{i+1}$ dimensional matrix.
 - So, a characterizing equation for $N_{i,j}$ is the following:

$$N_{i,j} = \min_{i \leq k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\}$$

- ◆ Note that sub-problems are not independent
→ the **sub-problems overlap**.

A Dynamic Programming Algorithm

Algorithm *matrixChain()*:

for $i \leftarrow 1$ **to** $n-1$ **do**

$N_{i,i} \leftarrow 0$

for $b \leftarrow 1$ **to** $n-1$ **do**

for $i \leftarrow 0$ **to** $n-b-1$ **do**

$j \leftarrow i+b$

$N_{i,j} \leftarrow \infty$

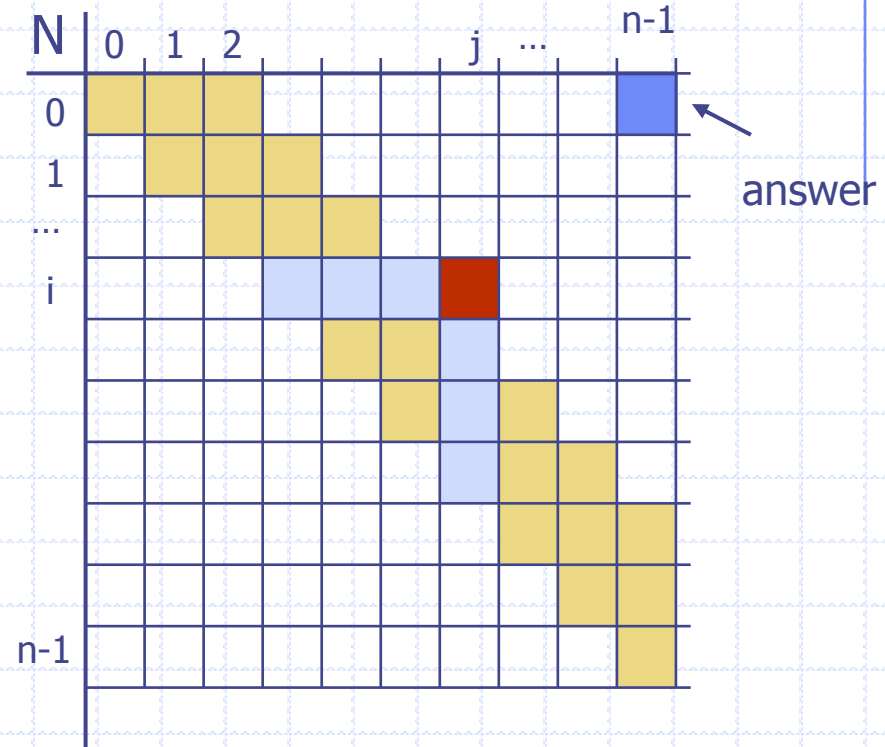
for $k \leftarrow i$ **to** $j-1$ **do**

$N_{i,j} \leftarrow \min\{N_{i,j}, N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\}$ and $S[i,j] \leftarrow k$

- ◆ Since **sub-problems overlap**, we don't use recursion. But, we construct optimal sub-problems "bottom-up."
 - ◆ $N_{i,i}$'s are easy, so start with them
 - ◆ Then do length 2,3,... sub-problems, and so on.
- Running time: $O(n^3)$

A Dynamic Programming Algorithm Visualization

- ◆ The bottom-up construction fills in the N array by diagonals
- ◆ $N_{i,j}$ gets values from previous entries in i-th row and j-th column
- ◆ Filling in each entry in the N table takes $O(n)$ time.
- ◆ Total run time: $O(n^3)$
- ◆ Getting actual parenthesization can be done by remembering "k" for each N entry



$$N_{i,j} = \min_{i \leq k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\}$$

Recursive Matrix Chain

Algorithm *RecursiveMatrixChain*(A, i, j):

if $i = j$ **return** 0

$N_{i,j} \leftarrow \infty$

for $k \leftarrow 1$ **to** $j-1$ **do**

$q \leftarrow \text{RecursiveMatrixChain}(A, i, k) +$

$\text{RecursiveMatrixChain}(A, k+1, j) + d_i d_{k+1} d_{j+1}$

if $q < N_{i,j}$ **then** $N_{i,j} \leftarrow q$

return $N_{i,j}$

◆ $T(1) \geq 1,$

◆ $T(n) \geq \sum [T(k) + T(n-k) + 1] + 1$ for $n > 1$ // $k=1$ to $n-1$

$\geq 2^{n-1}$

Memoization Matrix Chain

Algorithm *MemoizedMatrixChain()*:

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$N_{i,j} \leftarrow \infty$

return **LookupChain**($A, 1, n$)

Algorithm *LookupChain* (A, i, j):

if $N_{i,j} < \infty$ **return** $N_{i,j}$

if $i = j$

$N_{i,j} \leftarrow 0$

else

for $k \leftarrow 1$ **to** $j-1$ **do**

$q \leftarrow \text{LookupChain}(A, i, k) + \text{LookupChain}(A, k+1, j) + d_i d_{k+1} d_{j+1}$

if $q < N_{i,j}$ **then** $N_{i,j} \leftarrow q$

return $N_{i,j}$

Memoization Matrix Chain

- ◆ **LookUpChain()** always returns the value of $N_{i,j}$, but it only computes if this is the first time that **LookUpChain()** has been called with the parameters i and j .
- ◆ Therefore, **MemoizedMatrixChain()** saves time over, **RecursiveMatrixChain()**.
- ◆ Matrix chain multiplication problem can be solved by either a top down memoized algorithm or a bottom up dynamic programming

What are time efficiency ?

The General Dynamic Programming Technique

- ◆ Applies to a problem that at first seems to require a lot of time (possibly exponential), provided we have:
 - **Simple subproblems:** the sub-problems can be defined in terms of a few variables, such as j , k , l , m , and so on.
 - **Subproblem optimality:** the global optimum value can be defined in terms of optimal sub-problems
 - **Subproblem overlap:** the sub-problems are not independent, but instead they overlap (hence, should be constructed bottom-up).

The 0/1 Knapsack Problem

- ◆ Given: A set S of n items, with each item i having
 - b_i - a positive benefit
 - w_i - a positive weight
- ◆ Goal: Choose items with maximum total benefit but with weight at most W .
- ◆ If we are **not** allowed to take fractional amounts, then this is the **0/1 knapsack problem**.
 - In this case, we let T denote the set of items we take

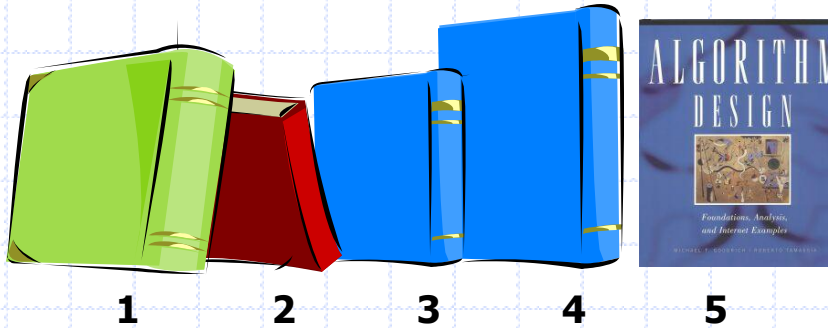
- Objective: maximize
$$\sum_{i \in T} b_i$$

- Constraint:
$$\sum_{i \in T} w_i \leq W$$

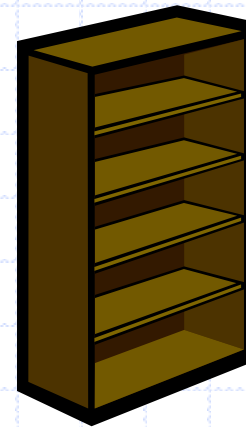
Example

- ◆ Given: A set S of n items, with each item i having
 - b_i - a positive benefit
 - w_i - a positive weight
- ◆ Goal: Choose items with maximum total benefit but with weight at most W .

Items:



Weight:	4 in	2 in	2 in	6 in	2 in
Benefit:	\$20	\$3	\$6	\$25	\$80



"knapsack"

Solution:

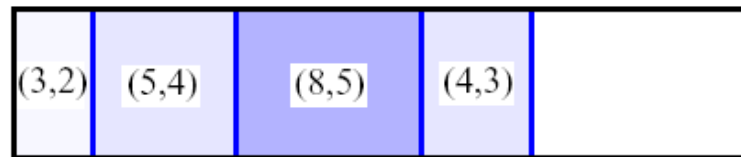
- 5 (2 in)
- 3 (2 in)
- 1 (4 in)

9 in

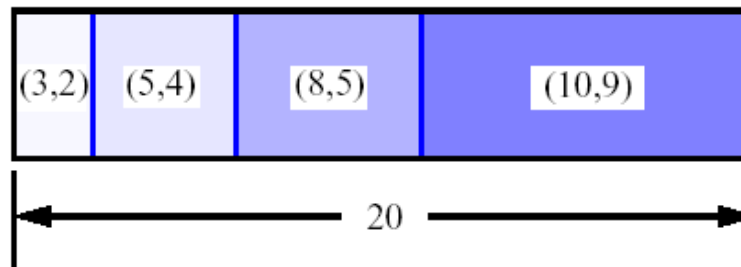
A 0/1 Knapsack Algorithm, 1st Attempt

- ◆ S_k : Set of items numbered 1 to k .
- ◆ Define $B[k]$ = best selection from S_k .
- ◆ Problem: does not have sub-problem optimality:
 - Consider $S = \{(3,2), (5,4), (8,5), (4,3), (10,9)\}$ weight-benefit pairs

Best for S_4 :



Best for S_5 :



A 0/1 Knapsack Algorithm, 2nd Attempt

- ◆ S_k : Set of items numbered 1 to k .
- ◆ Define $B[k, w]$ = best selection from S_k with weight exactly equal to w
- ◆ Good news: this does have subproblem optimality:

$$B[k, w] = \begin{cases} B[k-1, w] & \text{if } w_k > w \\ \max\{ B[k-1, w], B[k-1, w - w_k] + b_k \} & \text{else} \end{cases}$$

- ◆ I.e., best subset of S_k with weight exactly w is either the best subset of S_{k-1} w/ weight w or the best subset of S_{k-1} with weight $w - w_k$ plus item k .

The 0/1 Knapsack Algorithm

Algorithm *01Knapsack*(S, W):

// Input: set S of items w/ benefit b_i and weight w_i ; max. weight W

// Output: benefit of best subset with weight at most W

for $w \leftarrow 0$ **to** W **do**

$B[w] \leftarrow 0$

for $k \leftarrow 1$ **to** n **do**

for $w \leftarrow W$ **downto** w_k **do**

if $B[w-w_k] + b_k > B[w]$ **then**

$B[w] \leftarrow B[w-w_k] + b_k$

- ◆ Since $B[k, w]$ is defined in terms of $B[k-1, *]$, we can reuse the same array
- ◆ Running time: $O(nW)$.
- ◆ Not a polynomial-time algorithm if W is large, this is a **pseudo-polynomial** time algorithm

Longest Common Subsequence (LCS)

- ◆ Given a sequence $X = \langle x_1, x_2, \dots, x_m \rangle$ another sequence $Z = \langle z_1, z_2, \dots, z_k \rangle$ is a **subsequence** of X if there exists a strictly increasing sequence $\langle i_1, i_2, \dots, i_k \rangle$ of indices of X such that for all $j = 1, 2, \dots, k$, we have $x_{i_j} = z_j$.
- ◆ $Z = \langle B, C, D, B \rangle$ is a subsequence of $X = \langle A, B, C, B, D, A, B \rangle$ with the corresponding index sequence $\langle 2, 3, 5, 7 \rangle$.
- ◆ Given two sequences of X and Y , we say that a sequence Z is a **common subsequence** of X and Y if Z is a subsequence of both X and Y .
- ◆ $X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$, the sequence $\langle B, C, A \rangle$ is a common subsequence of both X and Y but the **longest common subsequence** is $\langle B, C, B, A \rangle$ or $\langle B, D, A, B \rangle$.

Longest Common Subsequence (LCS)

◆ In, the LCS problem, we are given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ and wish to find a maximum length common subsequence of X and Y .

◆ The ***Optimal-Substructure of an LCS Theorem*** is

Let $X = \langle x_1, \dots, x_m \rangle$ and $Y = \langle y_1, \dots, y_n \rangle$ be sequences, and let $Z = \langle z_1, \dots, z_k \rangle$ be any LCS of X and Y .

- If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .
- If $x_m \neq y_n$ and $z_k \neq x_m$, then Z is an LCS of X_{m-1} and Y .
- If $x_m \neq y_n$ and $z_k \neq y_n$, then Z is an LCS of X and Y_{n-1} .

A recursive solution to sub-problems

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

- ◆ $C[i, j]$ is the length of an LCS of the sequences X_i and Y_j .
If either $i=0$ or $j=0$, one of the sequences has length 0, so the LCS has length 0.

Compute the LCS

Algorithm *LCS*(X, Y):

for $w \leftarrow 0$ **to** W **do**

$m \leftarrow \text{LENGTH}[X]; n \leftarrow \text{LENGTH}[Y]$

for $i \leftarrow 1$ **to** m **do** $c[i, 0] \leftarrow 0$

for $j \leftarrow 0$ **to** n **do** $c[0, j] \leftarrow 0$

$b \leftarrow c$

for $i \leftarrow 1$ **to** m **do**

for $j \leftarrow 1$ **to** n **do**

if $(x_i = y_j)$

$c[i, j] \leftarrow c[i-1, j-1] + 1; b[i, j] \leftarrow \text{"UP-LEFT"}$

else if

$(c[i-1, j] \geq c[i, j-1]) \quad c[i, j] \leftarrow c[i-1, j]; b[i, j] \leftarrow \text{"UP"}$

else

$c[i, j] \leftarrow c[i, j-1]; b[i, j] \leftarrow \text{"LEFT"}$

return c and b

LCS problem

- ◆ $C[i,j]$ is the length of an LCS of the sequences X_i and Y_j .
If either $i=0$ or $j=0$, one of the sequences has length 0, so the LCS has length 0.

		0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	1	1	1	1	2	2
3	C	0	1	1	2	2	2	2
4	B	0	1	1	2	2	3	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	4
7	B	0	1	2	2	3	4	4

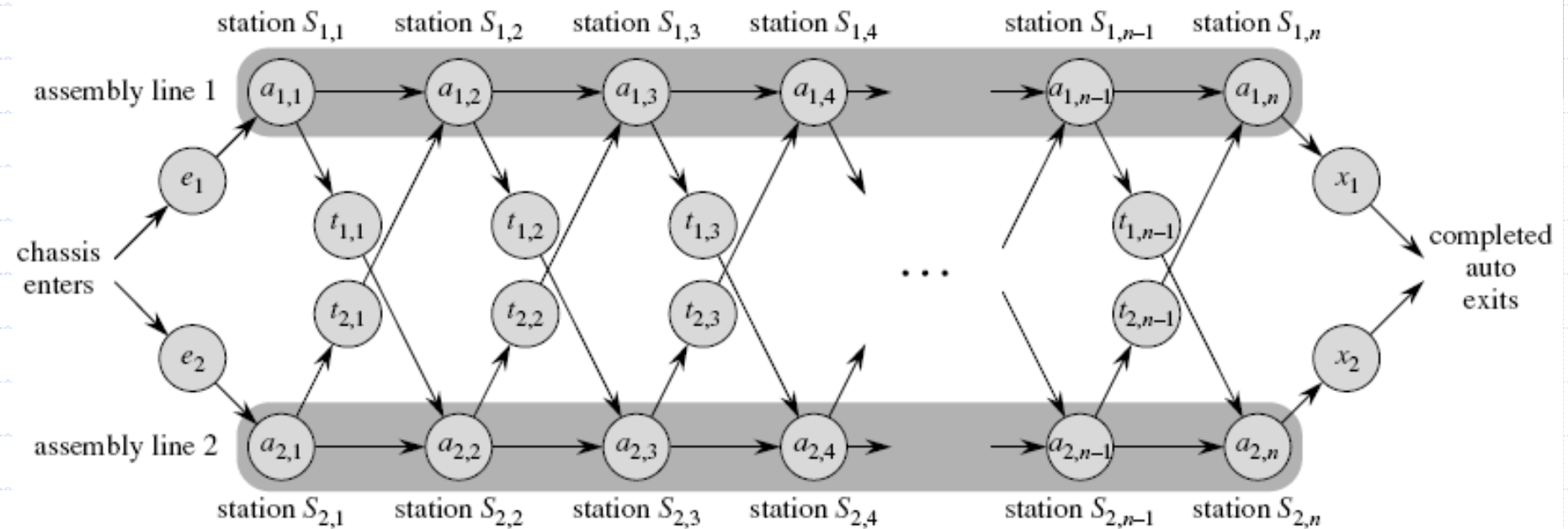
Compute the LCS

- ◆ The algorithm creates a table c and with the lengths of X and Y .
- ◆ The algorithm also has a table b , which is a copy of c , which is used to store the optimal solution of the LCS.
- ◆ The algorithm runs in $O(mn)$ time, where m and n are the length of X and Y respectively.

Constructing the LCS

- ◆ Starting from the (m,n) position, follow the direction of the arrows, until reach the end of the LCS.
- ◆ The values in c-table corresponding to the "UP-LEFT" positions in the b-table would be an element of LCS.
- ◆ The LCS is constructed in reverse order using this method. The construction step takes $O(m + n)$ time to complete, therefore it runs in linear time.

Assembly-Line Scheduling



Assembly-Line Scheduling

- ◆ A manufacturing problem to find the fastest way through a factory.
- ◆ There are two assembly lines, each with n stations; the j^{th} station on line i is denoted $S_{i,j}$ and the assembly time at that station is $a_{i,j}$. An automobile chassis enters the factory, and goes onto line i (where $i = 1$ or 2), taking e_i time.
- ◆ After going through the j^{th} station on a line, the chassis goes on to the $(j+1)^{\text{st}}$ station on either line. There is no transfer cost if it stays on the same line, but it takes time $t_{i,j}$ to transfer to the other line after station $S_{i,j}$. After exiting the n^{th} station on a line, it takes x_i time for the completed auto to exit the factory.
- ◆ The problem is to determine which stations to choose from line 1 and which to choose from line 2 in order to minimize the total time through the factory for one auto.

Brute Force Solution

- ◆ List all possible sequences,
- ◆ For each sequence of n stations, compute the passing time. (the computation takes $\Theta(n)$ time.)
- ◆ Record the sequence with smaller passing time.
- ◆ However, there are total 2^n possible sequences.

ALS --DP steps: Step 1

- ◆ Step 1: find the structure of the fastest way through factory
 - Consider the fastest way from starting point through station $S_{1,j}$ (same for $S_{2,j}$)
 - ◆ $j=1$, only one possibility
 - ◆ $j=2,3,\dots,n$, two possibilities: from $S_{1,j-1}$ or $S_{2,j-1}$
 - from $S_{1,j-1}$, additional time $a_{1,j}$
 - from $S_{2,j-1}$, additional time $t_{2,j-1} + a_{1,j}$
 - ◆ suppose the fastest way through $S_{1,j}$ is through $S_{1,j-1}$, then the chassis must have taken a fastest way from starting point through $S_{1,j-1}$.
 - ◆ Similarly for $S_{2,j-1}$.

DP step 1: Find Optimal Structure

- ◆ An optimal solution to a problem contains within it an optimal solution to subproblems.
- ◆ the fastest way through station $S_{i,j}$ contains within it the fastest way through station $S_{1,j-1}$ or $S_{2,j-1}$.
- ◆ Thus can construct an optimal solution to a problem from the optimal solutions to subproblems.

ALS --DP steps: Step 2

- ◆ Step 2: A recursive solution
- ◆ Let $f_i[j]$ ($i=1,2$ and $j=1,2,\dots, n$) denote the fastest possible time to get a chassis from starting point through $S_{i,j}$
- ◆ Let f^* denote the fastest time for a chassis all the way through the factory. Then
- ◆ $f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$
- ◆ $f_1[1] = e_1 + a_{1,1}$, fastest time to get through $S_{1,1}$
- ◆ $f_1[j] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j})$
- ◆ Similarly to $f_2[j]$.

ALS --DP steps: Step 2

◆ Recursive solution:

$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$

$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \text{if } j=1 \\ \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) & \text{if } j>1 \end{cases}$$

$$f_2[j] = \begin{cases} e_2 + a_{2,1} & \text{if } j=1 \\ \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}) & \text{if } j>1 \end{cases}$$

- ◆ $f_i[j]$ ($i=1,2; j=1,2,\dots,n$) records optimal values to the subproblems.
- ◆ To keep the track of the fastest way, introduce $l_i[j]$ to record the line number (1 or 2), whose station $j-1$ is used in a fastest way through $S_{i,j}$.
- ◆ Introduce f^* to be the line whose station n is used in a fastest way through the factory.

ALS --DP steps: Step 3

◆ Step 3: Computing the fastest time

- One option: a recursive algorithm.
 - ◆ Let $r_i(j)$ be the number of references made to $f_i[j]$
 - $r_1(n) = r_2(n) = 1$
 - $r_1(j) = r_2(j) = r_1(j+1) + r_2(j+1)$
 - $r_i(j) = 2^{n-j}$.
 - So $f_1[1]$ is referred to 2^{n-1} times.
 - Total references to all $f_i[j]$ is $\Theta(2^n)$.
 - ◆ Thus, the running time is exponential.
- Non-recursive algorithm.

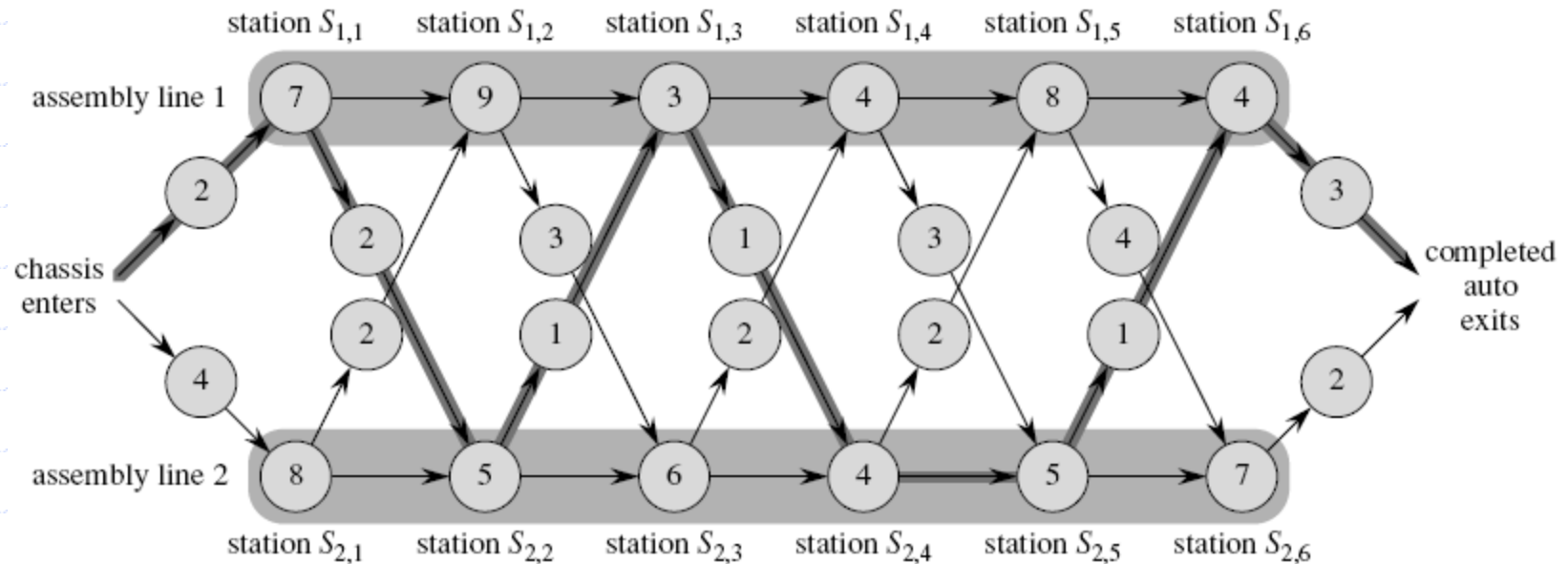
ALS Fast-way algorithm

FASTEST-WAY(a, t, e, x, n)

```
1   $f_1[1] \leftarrow e_1 + a_{1,1}$ 
2   $f_2[1] \leftarrow e_2 + a_{2,1}$ 
3  for  $j \leftarrow 2$  to  $n$ 
4      do if  $f_1[j-1] + a_{1,j} \leq f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
5          then  $f_1[j] \leftarrow f_1[j-1] + a_{1,j}$ 
6               $l_1[j] \leftarrow 1$ 
7          else  $f_1[j] \leftarrow f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
8               $l_1[j] \leftarrow 2$ 
9      if  $f_2[j-1] + a_{2,j} \leq f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
10         then  $f_2[j] \leftarrow f_2[j-1] + a_{2,j}$ 
11              $l_2[j] \leftarrow 2$ 
12         else  $f_2[j] \leftarrow f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
13              $l_2[j] \leftarrow 1$ 
14 if  $f_1[n] + x_1 \leq f_2[n] + x_2$ 
15     then  $f^* = f_1[n] + x_1$ 
16          $l^* = 1$ 
17 else  $f^* = f_2[n] + x_2$ 
18      $l^* = 2$ 
```

Running time: $\mathcal{O}(n)$.

ALS Fast-way algorithm



(a)

j	1	2	3	4	5	6
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37

$f^* = 38$

j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2

$l^* = 1$

Acknowledgement

- ◆ <http://www.personal.kent.edu/~rmuhamma/Algorithms/algorithm.html>
- ◆ <http://ww3.algorithmdesign.net/>
- ◆ <http://en.wikipedia.org/wiki/>
- ◆ <http://www.cse.unsw.edu.au/~lambert/java/3d/hull.html?dimension=2D&model=Divide+and+Conquer&seed=24513&npoints=20&frame=37>